

```
//Shafin Ahmed 232-15-184 65-M1 (Lab 03)

#include <GL/gl.h>

#include <GL/glut.h>

/* everything inside the function display() appears on the
screen. */

void display(void)
{

/* glClear() clears the screen using the background color specified earlier */
/* GL_COLOR_BUFFER_BIT means clear the color buffer (screen) */

glClear (GL_COLOR_BUFFER_BIT);

/* basically erase all previously drawn colors and start fresh. */


///1st body //Pink

float Red=255;
float Green=3;
float Blue=200;

glColor3f (Red/255,Green/255, Blue/255); /* glColor3f(R, G, B) */

// glColor3f(1,0,0); // red
// glColor3f(0,1,0); // green
// glColor3f(0,0,1); // blue
// glColor3f(1,1,1); // white
// glColor3f(0,0,0); // black


glBegin(GL_POLYGON);
/* OpenGL connects all vertices in order and
fills the shape */
glVertex3f (153, 142, 0.0);

glVertex3f (156, 134, 0.0);

glVertex3f (159, 142, 0.0);
```

```
glVertex3f (167, 145, 0.0);

glVertex3f (159, 148, 0.0);

glVertex3f (156, 156, 0.0);

glVertex3f (153, 148, 0.0);

glVertex3f (145, 145, 0.0);

glEnd();

///5th body //Komola

Red=245;
Green=73;
Blue=39;

glColor3f (Red/255,Green/255, Blue/255);

glBegin(GL_POLYGON);

glVertex3f (31, 142, 0.0);
glVertex3f (34, 134, 0.0);

glVertex3f (37, 142, 0.0);

glVertex3f (45, 145, 0.0);

glVertex3f (37, 148, 0.0);


glVertex3f (34, 156, 0.0);

glVertex3f (31, 148, 0.0);

glVertex3f (23, 145, 0.0);

glEnd();

///2nd body //Beguni

Red=152;
Green=23;
Blue=227;
```

```
glColor3f (Red/255,Green/255, Blue/255);
```

```
glBegin(GL_POLYGON);
```

```
glVertex3f (131, 142, 0.0);
```

```
glVertex3f (134, 134, 0.0);
```

```
glVertex3f (137, 142, 0.0);
```

```
glVertex3f (145, 145, 0.0);
```

```
glVertex3f (137, 148, 0.0);
```

```
glVertex3f (134, 156, 0.0);
```

```
glVertex3f (131, 148, 0.0);
```

```
glVertex3f (123, 145, 0.0);
```

```
glEnd();
```

```
///6th body //LaI
```

```
Red=227;
```

```
Green=23;
```

```
Blue=37;
```

```
glColor3f (Red/255,Green/255, Blue/255);
```

```
glBegin(GL_POLYGON);
```

```
/* OpenGL connects all vertices in order and  
fills the shape */
```

```
glVertex3f (253, 142, 0.0);
```

```
glVertex3f (256, 134, 0.0);
```

```
glVertex3f (259, 142, 0.0);
```

```
glVertex3f (267, 145, 0.0);
```

```
glVertex3f (259, 148, 0.0);
```

```
glVertex3f (256, 156, 0.0);
```

```
glVertex3f (253, 148, 0.0);

glVertex3f (245, 145, 0.0);

glEnd();


///3rd body //Sobuj

Red=250;
Green=233;
Blue=22;

glColor3f (Red/255,Green/255, Blue/255);

glBegin(GL_POLYGON);

glVertex3f (142, 153, 0.0);

glVertex3f (145, 145, 0.0);

glVertex3f (148, 153, 0.0);

glVertex3f (156, 156, 0.0);

glVertex3f (148, 159, 0.0);

glVertex3f (145, 167, 0.0);

glVertex3f (142, 159, 0.0);

glVertex3f (134, 156, 0.0);

glEnd();


///7th body //Sobuj

Red=167;
Green=250;
Blue=22;

glColor3f (Red/255,Green/255, Blue/255);

glBegin(GL_POLYGON);
```

```
glVertex3f (142, 253, 0.0);  
glVertex3f (145, 245, 0.0);  
glVertex3f (148, 253, 0.0);  
glVertex3f (156, 256, 0.0);  
glVertex3f (148, 259, 0.0);  
glVertex3f (145, 267, 0.0);  
glVertex3f (142, 259, 0.0);  
glVertex3f (134, 256, 0.0);  
  
glEnd();  
  
///4th body //Brown  
  
Red=176;  
Green=96;  
Blue=53;  
  
glColor3f (Red/255,Green/255, Blue/255);  
  
glBegin(GL_POLYGON);  
glVertex3f (142, 131, 0.0);  
glVertex3f (145, 123, 0.0);  
glVertex3f (148, 131, 0.0);  
glVertex3f (156, 134, 0.0);  
glVertex3f (148, 137, 0.0);  
glVertex3f (145, 145, 0.0);  
glVertex3f (142, 137, 0.0);  
glVertex3f (134, 134, 0.0);  
  
glEnd();
```

```
///8th body //Nil
```

```
Red=31;
```

```
Green=126;
```

```
Blue=237;
```

```
glColor3f (Red/255,Green/255, Blue/255);
```

```
glBegin(GL_POLYGON);
```

```
glVertex3f (142, 31, 0.0);
```

```
glVertex3f (145, 23, 0.0);
```

```
glVertex3f (148, 31, 0.0);
```

```
glVertex3f (156, 34, 0.0);
```

```
glVertex3f (148, 37, 0.0);
```

```
glVertex3f (145, 45, 0.0);
```

```
glVertex3f (142, 37, 0.0);
```

```
glVertex3f (134, 34, 0.0);
```

```
glEnd();
```

```
/* start processing buffered OpenGL routines */
```

```
glFlush ();
```

```
}
```

```
void init (void)
```

```
{
```

```
/* clearing (background) color / sets it to black */
```

```
glClearColor (0.0, 0.0, 0.0, 0.0);
```

```

/* initialize viewing values or configures the camera */
glMatrixMode(GL_PROJECTION);

glLoadIdentity(); //resets any previous transformation

glOrtho(0.0, 300.0, 0.0, 300.0,-1.0, 1.0);
// creates 2D coordinate space
/* This treats the window as a graph */

}

/*
*      Declare initial window size, position, and display mode
*      (single buffer and RGBA). Open window with "hello"
*      in its title bar. Call initialization routines.
*      Register callback function to display graphics.
*      Enter main loop and process events.
*/
int main(int argc, char** argv)
{
    glutInit(&argc, argv); // glut initialization

    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
    // single buffering and RGB colors

    glutInitWindowSize (700, 700);
    // size 500x500

    glutInitWindowPosition (100, 5);
    // 100 pixels from left
    // 100 pixels from top

    glutCreateWindow ("Star Designing");

```

```
init ();  
  
glutDisplayFunc(display);  
  
glutMainLoop();  
  
/* without this line the window would immediately close */  
  
return 0; /* ISO C requires main to return int. */  
}
```

Expected Output:

